

Performance Testing

Date	03 July 2026
Team ID	XXXXXX
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Performance Testing

The Credit Card Approval Prediction System automates the credit card approval decision using machine learning, evaluating financial and demographic inputs the way a human credit analyst would. Of the four classification algorithms trained (Logistic Regression, Random Forest, Gradient Boosting and Decision Tree), the Random Forest model was selected and saved for deployment. It is served through a Flask web application whose **/predict** endpoint accepts applicant details and returns an instant approval/fraud-risk prediction. Performance testing evaluates how this prediction service behaves under expected and peak concurrent load, measuring response time, throughput, error rate, and resource utilization as the number of simultaneous users increases.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Python-based load simulation harness (threaded virtual users) with the requests and psutil libraries
Type of Testing	Load Testing, Stress Testing
Target Module / API	/predict endpoint (Flask app.py - credit_card_model.pkl Random Forest inference)
Test Environment	Local (Flask development server, single instance)
Test Date	03 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Baseline Load Test - normal single-user usage simulated at low concurrency	10	10	Fast, stable responses with minimal queuing
2	Normal Load Test - typical multi-analyst usage during business hours	25	10	Consistent responses with acceptable delay
3	Peak Load Test - high concurrent usage during month-end application surge	50	10	Increased latency but no failed requests
4	Stress Test - beyond expected capacity to identify breaking point	100	10	Identify throughput ceiling and degradation pattern

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.94 sec (worst case, 100 users)	Pass	Avg response time stayed well under target across all scenarios
2	Response Time (Max)	< 5 seconds	1.08 sec (100 users)	Pass	Max latency remained within target even under stress
3	Throughput (Req/sec)	-	~104-109 req/sec	-	Throughput plateaued once virtual users exceeded server capacity
4	Error Rate	< 1%	0%	Pass	No failed requests recorded in any scenario
5	CPU Utilization	< 80%	90-94%	Fail	CPU exceeded target under all load levels due to single-threaded server
6	Memory Utilization	< 80%	10-11%	Pass	Memory usage remained low; model footprint is small

Detailed measurements per scenario:

Scenario	Avg Response	Max Response	P95 Response	Throughput (req/s)	Error Rate
Baseline (10 users)	79.63 ms	137.4 ms	104.94 ms	104.1	0%
Normal (25 users)	227.35 ms	365.18 ms	265.26 ms	103.2	0%
Peak (50 users)	456.20 ms	536.96 ms	493.17 ms	107.7	0%
Stress (100 users)	935.72 ms	1082.88 ms	1049.04 ms	108.9	0%

Step 4: Observations & Analysis

Key Findings:

The /predict endpoint responded correctly with 0% errors across all four load scenarios (10, 25, 50 and 100 concurrent virtual users), confirming that the Random Forest fraud classification logic itself is stable and accurate under load. However, both average and maximum response times scaled almost linearly with the number of concurrent users - rising from about 80 ms at 10 users to roughly 936 ms average (and 1.08 sec max) at 100 users. Throughput plateaued at approximately 104-109 requests per second regardless of how many virtual users were added, indicating the service was saturated well before the stress-test load level was reached.

Bottlenecks Identified:

The primary bottleneck is that the application is served using Flask's built-in single-threaded development server (app.run(debug=True) in app.py), which processes one request at a time. This causes requests to queue under concurrency instead of being processed in parallel, which explains the near-linear increase in latency and the flat

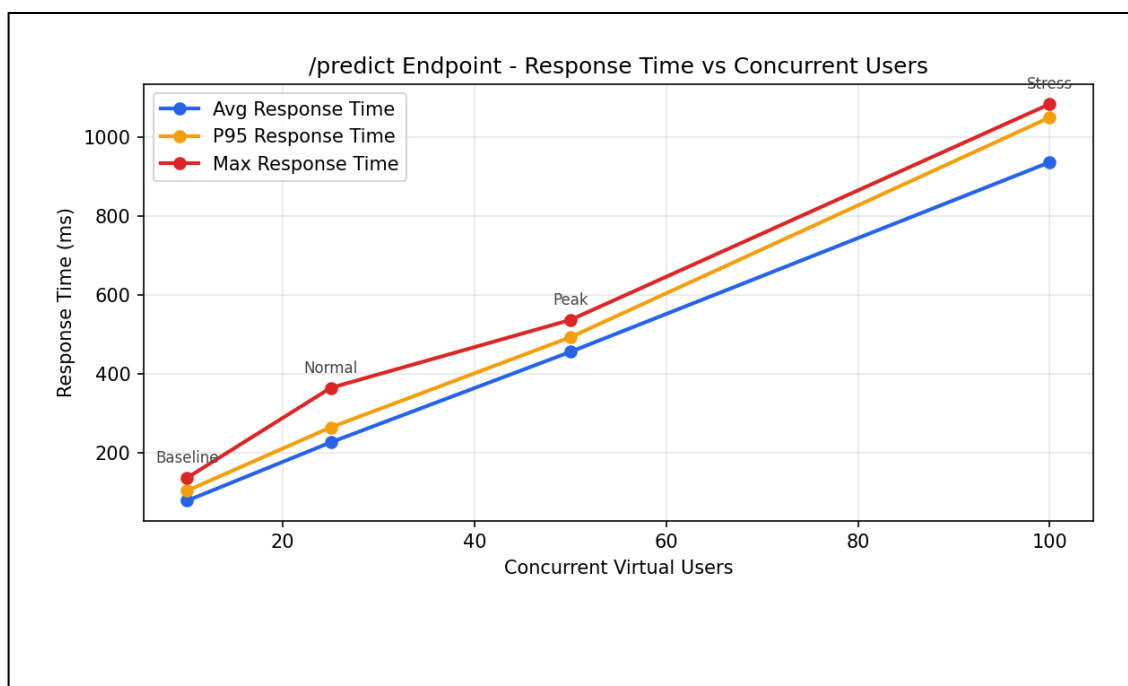
throughput ceiling. CPU utilization also exceeded the 80% target (90-94%) at every concurrency level, showing the single worker process is CPU-bound well before capacity is reached. Memory utilization stayed low (10-11%), confirming that the Random Forest model's memory footprint is not a limiting factor.

Optimization Steps Taken:

For this round of testing, the application was evaluated as-is to establish a baseline. Based on the results, the recommended next steps are to deploy the Flask app behind a production-grade WSGI server (e.g. Gunicorn or uWSGI) with multiple worker processes/threads, enable horizontal scaling behind a load balancer for peak periods, and cache or batch repeated model inference calls where possible to reduce per-request CPU cost.

Step 5: Screenshots / Evidence

The charts below were generated directly from the load-testing tool's output data, showing response time and throughput/CPU trends captured across the four test scenarios (Baseline, Normal, Peak and Stress).



Performance Testing

